SpeakSport AI Receptionist: The Ultimate Customer Success & Prompting Guide

Welcome to the SpeakSport AI Management SOP.

This document contains the exact, step-by-step procedures required to manage, edit, and provision AI voice receptionists for our golf course facilities. While I am away, you will be the primary point of contact for executing prompt updates, configuring tools in Vapi, and onboarding new facilities.

It is absolutely critical that you follow these steps precisely as written. AI voice agents are highly sensitive to formatting, variable initialization, and tool configuration. A single missed step can result in the AI hallucinating prices, breaking the booking flow, or failing to transfer calls.

Table of Contents

---

1. Generating New Prompts for Newly Onboarded Facilities

Onboarding a new facility requires us to build a bespoke "System Prompt" from scratch. This prompt acts as the AI's brain, dictating its personality, knowledge, and operational logic.

We do not write these entirely by hand. Instead, we use Google AI Studio to generate them, using previous successful prompts as an architectural blueprint, and Firecrawl to scrape the facility's exact website data.

Step 1A: Select the Reference Prompt

Before generating anything, you must determine what *type* of facility we are onboarding. We primarily have two categories:

1. **Not-Integrated (SMS Based):** The AI does not have access to the tee sheet. It cannot book tee times directly. Instead, when a caller asks to book, the AI triggers an SMS tool to send the caller a link to the facility's online booking portal.
2. **Fully Integrated:** The AI connects directly to the tee sheet (like our Todd Creek setup). It can fetch inventory, check member passes, and book tee times directly over the phone.

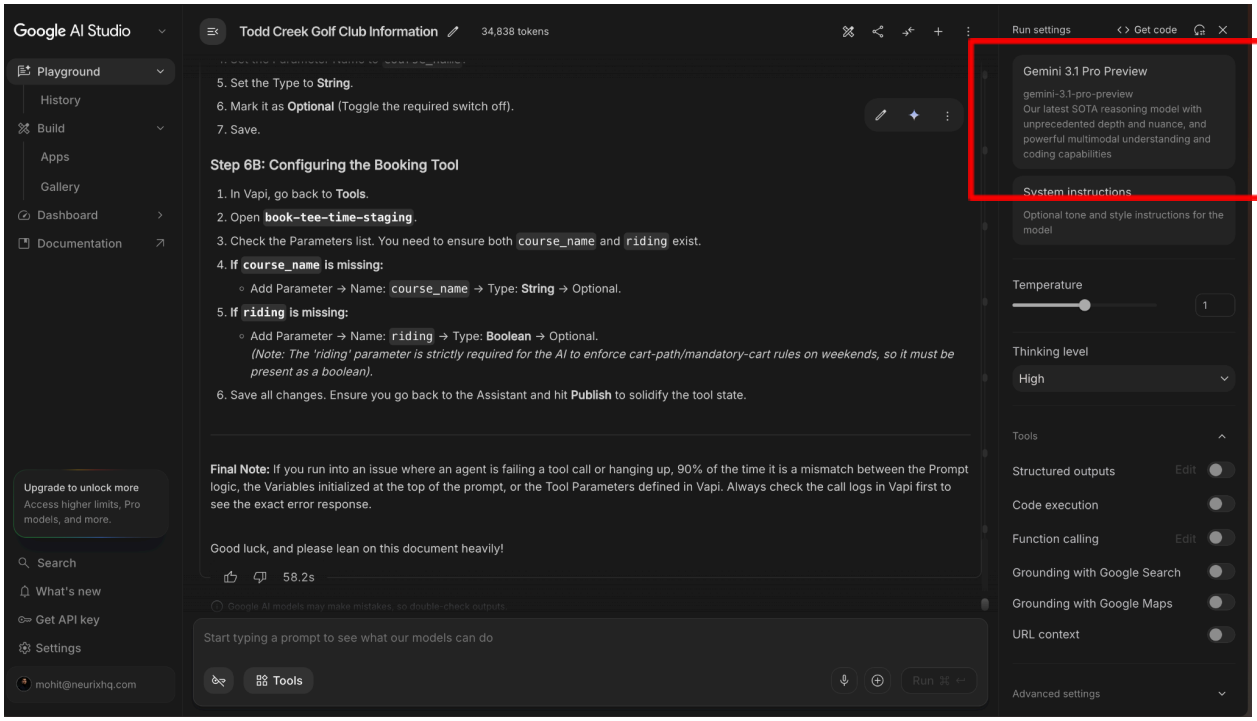
Below are the two reference prompts you will use depending on the facility type.

REFERENCE PROMPT 1 (NON-INTEGRATED SMS ONLY)

REFERENCE PROMPT 2 (FULLY INTEGRATED FLOW)

Step 1B: Prime Google AI Studio

1. Open a new chat in **Google AI Studio**.
2. **CRITICAL:** Ensure you select the **Gemini 3.1 PRO** (or the latest available Gemini Pro model) from the model picker in the top right corner. Do not use standard or flash models; they lack the reasoning capacity required for complex prompt engineering.



3. Copy the correct reference prompt from above.

4. Paste the reference prompt into the chat box, followed *exactly* by this instruction:

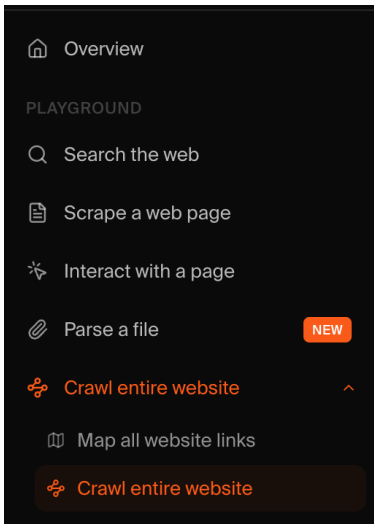
```
"HOLD ONTO THIS PROMPT. AWAIT FURTHER INSTRUCTION. IN THE MEANTIME, STUDY IT FOR ITS STRUCTURE, AND THE GOOD PRACTICES ESTABLISHED HERE SUCH AS CORE-SHELL, LOGIC-MODULE DISTINCTION, AND A BUNCH OF PROTOCOLS ESTABLISHED AROUND HOW TO TRANSFER, AND HOW TO RECOGNIZE PHONE NUMBERS. WE WILL USE THESE TO WRITE A PROMPT FOR A NEW CLIENT WE HAVE ONBOARDED. AWAIT FURTHER INSTRUCTIONS. FOR NOW, SIMPLY RESPOND WITH A YES IF YOU UNDERSTAND."
```

5. Submit the request. Wait for the AI to respond with a "Yes" or affirmative confirmation. Do not proceed until it has acknowledged this framework.

Step 1C: Extract the Facility Knowledge Base using Firecrawl

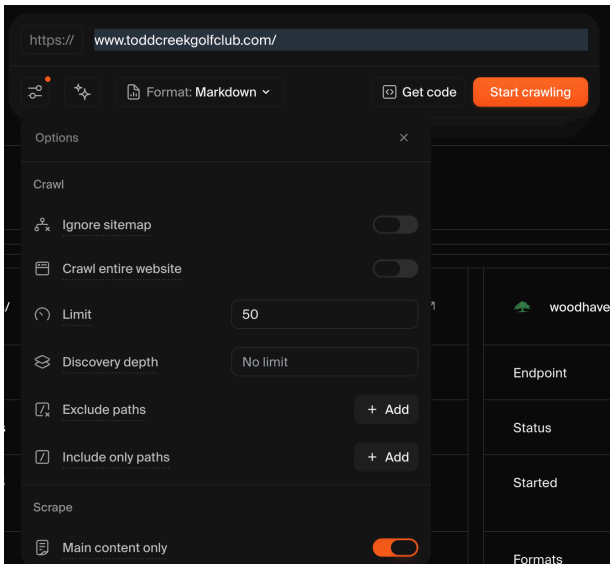
Now we need the facts. The AI needs to know the facility's hours, rates, dress codes, menus, and staff directories.

1. Go to **Firecrawl.dev**. This tool scrapes a website and converts it into AI-readable Markdown files.
2. Select the **"Crawl Entire Website"** option from the sidebar on the left.



3. Paste the URL of the new facility's website.

4. **Change the page limit to 50.** (The default is 10, which will cause you to miss sub-pages like the restaurant menu or driving range rules).



5. Run the crawl.
6. Once completed, download the files. This will download a `.zip` file to your computer.
7. Extract the `.zip` file. Inside, you will find multiple Markdown (`.md`) files, each representing a single page from the website.

Step 1D: Generate the New Prompt

Gather the Markdown files, along with any specific emails or onboarding notes from the client (e.g., special booking windows, custom rules not listed on the site).

Go back to your active Google AI Studio chat and paste the following master prompt. **Read it carefully before sending**, as you may need to adjust the transfer destinations based on the specific facility.

```
WE ARE GOING TO WRITE A PROMPT FOR A NEW FACILITY, WHILE USING THE PROMPT
SHARED EARLIER AS A REFERENCE FOR HOW WE WRITE PROMPTS AT SPEAKSPORT.

HERE IS ALL OF THE KNOWLEDGE BASE CONTENT I HAVE FOR THIS FACILITY. USE
THIS TO CRAFT A BESPOKE PROMPT FOR THIS FACILITY, WHILE KEEPING THE
PREVIOUS SPEAKSPORT PROMPT AS THE STYLE AND STRUCTURE REFERENCE.

IMPORTANT EXPECTATION

I expect you to maintain full detail from all knowledge base contents in
the prompt you write. I do not want a situation where the client tests the
receptionist and the AI is unable to answer something that was already
```

provided in the knowledge base.

Do not be stingy on token consumption. Use as many tokens as needed to output a strong, complete, production-ready prompt.

The final prompt should be detailed, practical, and operationally usable by a voice AI receptionist at a golf facility. It should include all important policies, FAQs, booking behavior, transfer behavior, facility details, and edge cases.

Use the previous SpeakSport prompt only as a reference for writing style, logical structure, and level of detail. Do not blindly copy irrelevant sections.

TRANSFER DESTINATIONS

Based on the facility context, create sensible transfer destinations.

When the receptionist needs to transfer the caller, instruct it to call the `transfer_call-staging` tool with `destination` set to the relevant destination.

Example:

Call the `transfer_call-staging` tool with destination = `events`.

Do not include phone numbers in the prompt. My transfer tool will handle the phone numbers by itself.

Create only destinations that make sense for this facility based on the knowledge base. Examples may include, but are not limited to:

- `golf_shop`
- `events`
- `restaurant`
- `instruction`
- `management`
- `membership`
- `maintenance`
- `lodging`
- `campground`

Only include destinations that are actually relevant to this facility.

CRITICAL TOOLING INSTRUCTIONS

Do not mess with the logical structure around how we use the tee time tools.

The facility prompt must correctly use the following tools:

1. check-booking-eligibility-staging
2. get-available-tee-times-staging
3. book-tee-time-staging
4. fetch-inventory, if relevant to this facility
5. transfer_call-staging

The booking flow must follow this order:

1. Understand the caller's request.
2. If the caller wants to book a tee time, first gather the requested date and time.
3. Once date and time are known, call check-booking-eligibility-staging before gathering the rest of the booking details.
4. If the caller is not eligible, stop the booking flow, explain the reason returned by the eligibility tool, and offer alternatives or a transfer.
5. If the caller is eligible, continue gathering players, holes, carts/riding preference, and any other required details.
6. Call get-available-tee-times to retrieve available inventory.
7. Present available times clearly.
8. Once the caller chooses a time, collect required booking details.
9. Call book-tee-time.
10. Confirm the booking clearly using the tool result.

DO NOT recreate booking eligibility logic manually inside the receptionist prompt. Eligibility is now handled by the check-booking-eligibility-staging tool.

VARIABLE INITIALIZATION REQUIREMENT

Ensure all runtime variables in curly braces are initialized near the top of the prompt before they are referenced anywhere else.

For example, initialize variables like this:

Caller Profile Context Variables:

Caller is customer: {{caller_is_customer}}
Customer Passes on File: {{customer_passes}}
Customer Groups on File: {{customer_groups}}
Customer Price Class: {{price_class}}
Customer has card on file: {{customer_has_card_on_file}}
Courses available: {{courses}}

After initialization, refer to the initialized variable names in the prompt logic, not the curly brace expressions.

Correct:

Use the initialized `Customer Passes on File` variable to understand whether the caller appears to have passes associated with their profile.

Incorrect:

Use `{{customer_passes}}` to understand whether the caller has a pass.

This is critical because curly brace variables are resolved at runtime. If you use the raw curly brace variable repeatedly in logic steps, the LLM may lose the intended semantic context.

Even though eligibility is handled by the tool, the receptionist prompt should still initialize caller profile variables so the AI has profile context for conversation, personalization, explanation, and escalation.

REVISED ELIGIBILITY TOOL ARCHITECTURE

We now have a `check-booking-eligibility-staging` tool.

This tool handles the task of checking whether someone is eligible to book a requested tee time.

The receptionist must call this tool as soon as it has the requested booking date and time, before gathering details such as group size, number of holes, carts, customer email, or searching actual tee time inventory.

The tool can accept the following optional parameters:

- `date`
- `time`

- num_holes
- course_name
- num_players

For this prompt generation, decide which of these parameters should be passed based on the facility's booking policies and the knowledge base provided.

At minimum, the receptionist must pass:

- date
- time

Format requirements:

- date must be formatted strictly as YYYY-MM-DD.
- time must be formatted strictly as 24-hour time, for example 14:00.

The backend automatically handles other context variables such as current date, customer passes, card on file, customer groups, and price class from the database based on the caller's phone number.

The tool returns a JSON payload containing:

- eligible: boolean true or false
- reason: human-readable sentence explaining the eligibility decision

If eligible is false:

- The receptionist must stop the booking flow.
- The receptionist must explain the reason provided by the tool in natural language.
- The receptionist may offer alternatives, such as another date or time, if appropriate.
- The receptionist may offer to transfer the caller to the golf shop if the situation requires human assistance.

If eligible is true:

- The receptionist should continue the booking flow.
- It may then ask for players, holes, carts/riding preference, and other booking details.

- It should then call `get-available-tee-times`.

BACKOFFICE POLICY PROMPT REQUIREMENT

In addition to the main receptionist prompt, create a separate Backoffice Policy Prompt for the eligibility tool.

This backoffice policy prompt is used by a backend reasoning model, Gemini 2.5 Flash Lite.

Write the backoffice policy prompt like a brief to a new front-desk employee.

It must be:

- Specific
- Direct
- Operational
- Easy for a reasoning model to apply
- Written as one rule per line wherever possible

It should reference logical criteria using natural language that maps clearly to backend properties such as:

- `date`
- `current_date`
- `customer_passes`
- `customer_has_card_on_file`
- `customer_groups`
- `price_class`
- `num_holes`
- `course_name`
- `num_players`

Do not write vague policy language. Convert the knowledge base into explicit eligibility rules.

For example:

Correct:

Public players may book tee times up to 7 days in advance.

Season pass holders with an active pass may book tee times up to 14 days in

advance.

A customer pass is active only if the pass exists and `expired: false`.

If a pass label says Mon-Fri, the pass should only be treated as valid for Monday through Friday.

If the requested date is outside the caller's booking window, return `eligible: false` and explain the earliest valid booking date.

Incorrect:

Customers can book depending on their pass.

MAIN BOOKING TOOL PARAMETERS

The `get-available-tee-times` tool requires the following parameters:

- `date`
  Type: string
  Required: yes
- `when`
  Type: string
  Required: yes
- `num_holes`
  Type: number
  Required: yes
- `course_name`
  Type: string
  Required: no
- `num_players`
  Type: number
  Required: yes

The `book-tee-time` tool requires the following parameters:

- `date`
  Type: string
  Required: yes
- `time`
  Type: string
  Required: yes
- `email`

Type: string
Required: yes

- riding
  Type: boolean
  Required: yes
- last_name
  Type: string
  Required: yes
- num_holes
  Type: number
  Required: yes
- first_name
  Type: string
  Required: yes
- course_name
  Type: string
  Required: no
- num_players
  Type: number
  Required: yes

COURSE NAME HANDLING

The course_name value must come from the initialized Courses available variable.

Initialize it like this:

Courses available: {{courses}}

When a course name is needed for check-booking-eligibility-staging, get-available-tee-times, or book-tee-time, instruct the receptionist to use the exact course name from the initialized Courses available variable.

Do not invent course names.

Do not hardcode course names unless they are explicitly present in the knowledge base and also represented in the initialized Courses available variable.

If there is only one course available, the receptionist may use that course automatically.

If there are multiple courses and the caller's requested course is ambiguous, ask a short clarification question.

If the caller does not care which course, choose the most appropriate course based on the facility's booking rules and availability flow.

GET-AVAILABLE-TEE-TIMES LOGIC

Do not change the core logic of the `get-available-tee-times` flow.

The receptionist must only call `get-available-tee-times` after the caller has passed eligibility checking.

Before calling `get-available-tee-times`, the receptionist needs:

- date
- general time window or requested time, passed as `when`
- number of holes
- number of players
- course name, if required or relevant

The `when` parameter should reflect the caller's desired time or time window naturally.

Examples:

- "morning"
- "afternoon"
- "around 9 AM"
- "after 2 PM"
- "before noon"
- "closest to 10:30 AM"

If the caller gives an exact time, use that exact time or a narrow natural-language equivalent in `when`.

If the caller is flexible, use a broader value such as "morning" or "afternoon."

BOOK-TEE-TIME LOGIC

Do not change the core logic of the `book-tee-time` flow.

Before calling book-tee-time, the receptionist must have:

- date
- exact selected tee time
- first name
- last name
- email
- riding preference
- number of holes
- number of players
- course name, if required or relevant

The receptionist should not ask for information it already has from the conversation unless confirmation is necessary.

The receptionist must clearly confirm the booking details before or after calling the booking tool, depending on the structure used in the reference prompt.

The receptionist should never claim a tee time is booked until the book-tee-time tool confirms success.

If booking fails, the receptionist should apologize, explain that the booking could not be completed, and offer to search another time or transfer to the golf shop.

FETCH INVENTORY

If the facility knowledge base includes merchandise, restaurant items, rental items, simulator bays, lessons, club fittings, or anything that requires inventory lookup, include the fetch-inventory tool in the prompt.

Do not include fetch-inventory if it is not relevant.

If included, explain exactly when the receptionist should call it and how it should present results.

VOICE AND BEHAVIOR

The receptionist should sound like a helpful, concise, professional golf course front-desk employee.

It should not sound robotic.

It should not over-explain policies unless the caller asks or the policy affects the caller's request.

It should ask one question at a time.

It should avoid long monologues on calls.

It should be warm, practical, and efficient.

It should gracefully handle interruptions, corrections, and caller frustration.

It should be able to answer facility questions directly from the knowledge base.

It should not hallucinate policies, prices, hours, booking windows, amenities, or phone numbers.

If the knowledge base does not contain an answer, the receptionist should say it does not have that information available and offer to transfer the caller to the appropriate department.

FACILITY KNOWLEDGE BASE REQUIREMENTS

Convert the facility knowledge base into a complete operational prompt.

Include all relevant details, including but not limited to:

- Facility overview
- Course names
- Number of holes
- Public vs private/semi-private access
- Booking windows
- Passholder/member rules
- Guest rules
- Card-on-file rules
- No-show or cancellation policies
- Rain check policies
- Cart policies
- Walking/riding policies
- Junior/senior/military/student pricing rules, if provided
- League/tournament/event rules
- Lesson/instruction details
- Driving range/practice facility details

- Restaurant/food and beverage details
- Pro shop details
- Rental club details
- Simulator details
- Dress code
- Pace of play
- Alcohol/smoking policies
- Weather/frost policies
- Holiday rules
- Accessibility rules
- Contact/transfer handling
- Any other FAQ content provided

Preserve nuance.

Do not compress away details just because they seem minor. If the client provided it, include it.

However, organize it cleanly so the receptionist can use it during live calls.

OUTPUT FORMAT

Your output should contain two main sections:

1. Main Voice AI Receptionist Prompt

This should be the full production prompt for the AI receptionist.

It should include:

- Role and identity
- Voice/tone instructions
- Runtime variable initialization
- General call handling rules
- Facility knowledge base
- Booking flow
- Eligibility tool usage
- Tee time search flow
- Tee time booking flow
- Transfer rules
- Tool usage rules
- Edge cases and failure handling

2. Backoffice Booking Eligibility Policy Prompt

This should be the separate policy prompt for the `check-booking-eligibility-staging` tool.

It should be written as clear operational rules, one rule per line wherever possible.

It should not include irrelevant receptionist behavior.

It should focus only on determining whether a requested booking is eligible.

REMEMBER

Do not manually implement booking eligibility logic in the main receptionist prompt.

Do include customer profile variables in the main receptionist prompt for context.

Do initialize all curly brace variables before referring to them.

Do refer to initialized variable names in logic, not raw curly brace variables.

Do preserve all relevant knowledge base detail.

Do use `check-booking-eligibility-staging` before `get-available-tee-times`.

Do use `get-available-tee-times` before offering actual available tee times.

Do use `book-tee-time` only after the caller selects an available tee time and required booking details are collected.

Do use exact course names from the initialized `Courses available` variable.

Do create sensible transfer destinations and call `transfer_call-staging` with `destination`.

Do not include transfer phone numbers.

```
Do not hallucinate missing policies.

Do not make the final prompt thin or generic.

Now generate the full facility-specific prompt using the knowledge base
below:

[PASTE FACILITY KNOWLEDGE BASE HERE]
```

[ATTACH ALL FIRECRAWL MARKDOWN FILES HERE] [PASTE ANY EXTRA CLIENT RULES/NOTES HERE]"

A Note on Runtime Variables: During the start of a call, our system injects a payload of variables into the prompt. You must understand what these are so you can explicitly tell the AI Studio model how to use them if the client has special rules. Here is an example of the payload:

JSON


```
{
  "email": "dclifton@toddcreekgolfclub.com",
  "courses": [ "Todd Creek" ],
  "zipcode": "80602",
  "greeting": "Thank you for calling the golf shop at Todd Creek. You’re speaking
with Birdie, our AI receptionist...",
  "last_name": "Clifton",
  "disclaimer": "",
  "first_name": "David",
  "booking_url": "",
  "price_class": "Public",
  "announcement": "",
  "current_date": "05/16/2026",
  "facility_gms": true,
  "facility_name": "Todd Creek Golf Club",
  "customer_groups": [ "Member" ],
  "customer_passes": [
    { "label": "Season Solo Mon-Sun", "expired": true },
    { "label": "Season Solo Mon-Fri", "expired": true }
  ],
  "phone_recognized": true,
  "price_class_names": [ "Club Play", "Comp", "Employee", "Public", etc... ],
  "caller_is_customer": true,
  "customer_is_member": true,
  "confirmation_sms_enabled": true,
  "customer_has_card_on_file": false,
```

```
  "booking_requires_card_on_file": false
}
```

Why this matters: If a facility tells you "Only active pass holders can book 14 days out, public is 7 days," you must explicitly tell Google AI Studio: *"Use the `customer_passes` variable to enforce a 14-day booking window if they have an active pass, otherwise 7 days."*

Step 1E: Deploying to Vapi & Backoffice Configuration

1. Once Google AI Studio generates the prompt, review it. Ensure no curly braces `{{ }}` are used inside the logic steps.
2. Click the three dots on the top right of the generated message and select **"Copy as Markdown"**.
3. Log into **Vapi**. Open the specific facility's workspace.
4. Navigate to the Assistant titled: `Facility Receptionist (staging)`.
5. Paste the entire Markdown prompt into the System Prompt box and click **Publish**.
6. Open the **SpeakSport Backoffice**. Navigate to the facility.
7. Ensure the `Greeting`, `Disclaimer`, and `Announcement` variables are correctly filled out according to the client's preference. Click **Save**.
8. **Call the staging number to test the agent.** Ensure it greets you properly and has accurate knowledge of the facility.

Step 1F: Version Control in Notion (CRITICAL)

We must maintain a strict history of our prompts.

1. Open **Notion** and go to the **Facilities and Clients DB**.
2. Open the page for this specific facility.
3. Scroll to the **Logic Module Versions** section.
4. Create a **Toggle Heading 3** (e.g., `> V1 - Initial Onboarding`).
5. Inside the toggle, insert a Code Block (`/code`) and paste the entire prompt.
6. For future updates (V2, V3, etc.), you will add *two* code blocks: one describing the exact changes made, and one containing the new prompt.

---

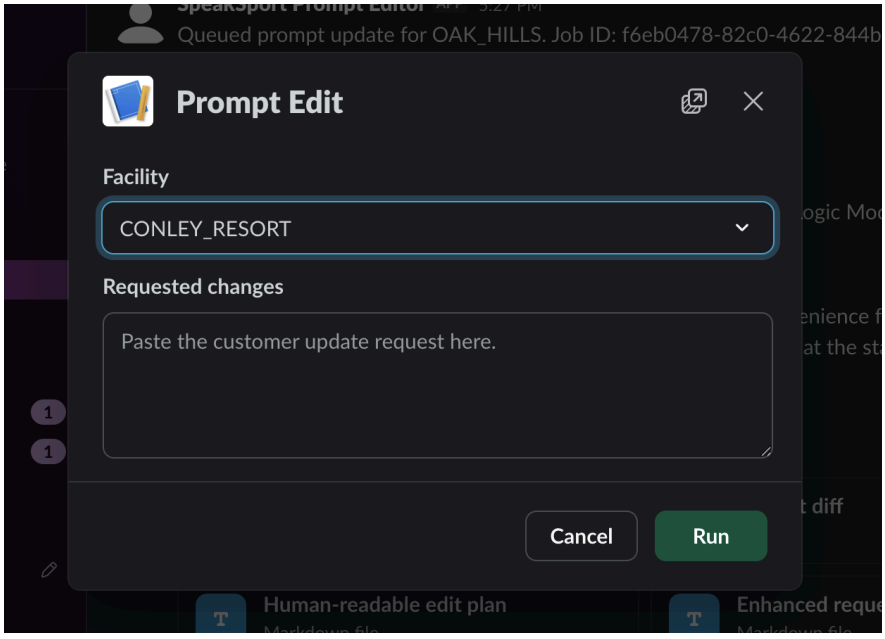
2. Editing Existing Prompts for Existing Facilities

When an existing client requests a change (e.g., "Change our Twilight rate to $45" or "Add a $2.50 phone booking fee"), **DO NOT rewrite the entire prompt.** Doing so risks deleting previous custom logic.

Instead, we use our custom **Slackbot AI Prompt Editor**. This tool uses an orchestration of 5 AI agents (Classifier, Enhancer, Planner, and Verifier) to safely generate surgical JSON patches to the prompt.

Step 2A: Triggering the Slackbot

1. Open Slack and navigate to the `#prompt-updates` channel.
2. Type the following command and hit enter: `/prompt-edit`
3. A dialogue box will appear. Select the specific Facility from the dropdown menu (**note: while I have added all of our existing facilities, new facilities won't be present as they require me to run a script on my side).**



4. In the text box, describe the exact changes the client requested. *Best Practice:* Even though the AI Enhancer will clean up your request, be as clear as possible. E.g., *"Change weekend twilight rates from $40 to $50. Also, add a rule to the booking flow that public guests must pay a $2.50 convenience fee for phone bookings."*
5. Submit the request.

Step 2B: Reviewing the Slackbot Output

Within 2 to 3 minutes, the Slackbot will reply in the channel. A successful job looks like this:

> **Queued prompt update for OAK_HILLS.** Job ID: f6eb0478-82c0-4622-844b-0a341429bf7d
>
> **Prompt update ready for review: OAK_HILLS** Status: Needs human review Job ID: f6eb0478-82c0-4622-844b-0a341429bf7d

Summary: Added a conditional disclaimer to the tee time booking flow in the Logic Module to inform non-members of a two dollar and fifty cent convenience fee for phone bookings.

Verifier: The prompt update successfully integrates the phone booking convenience fee disclaimer for non-members. TTS formatting and variable references are correct, and the logic is placed perfectly at the start of the booking flow.

Verifier issues: None reported by verifier.

Step 2C: Analyzing the Attached Files

Along with the message, the Slackbot will attach 4 files. You must review them:

1. **Enhanced Request Markdown:** Shows how the AI "Enhancer" cleaned up your raw request to make it safe for prompt editing (e.g., converting "$2.50" to "two dollars and fifty cents").
2. **Human Readable Edit Plan:** Shows the exact operations (`replace_section`, `insert_after`) the AI decided to take. We *never* rewrite; we only patch.
3. **Prompt Diff:** This is a GitHub-style code diff showing exactly what lines were added/removed from the live Vapi prompt.
4. **Updated System Prompt:** The final, completed prompt ready for deployment.

Step 2D: Checking the Diff & Deploying

1. Go to **PatchViewer**.
2. Upload the **Prompt Diff** file (or copy/paste its contents into the text box).
3. Review the side-by-side comparison. Ensure the AI didn't accidentally delete an unrelated section of the `<logic-module>`.
4. If the diff looks perfect, open the **Updated System Prompt** file.
5. Copy the entire contents.
6. Go to **Vapi**, open the facility's `Facility Receptionist (staging)` agent, paste the new prompt, and hit **Publish**.
7. **Notion Versioning:** Go to Notion, create a new Toggle (e.g., `> V2 - Rate Update & Convenience Fee`), paste the summary of changes in one code block, and the new prompt in the second code block.

---

3. Configuring Transfers on Vapi for Facilities

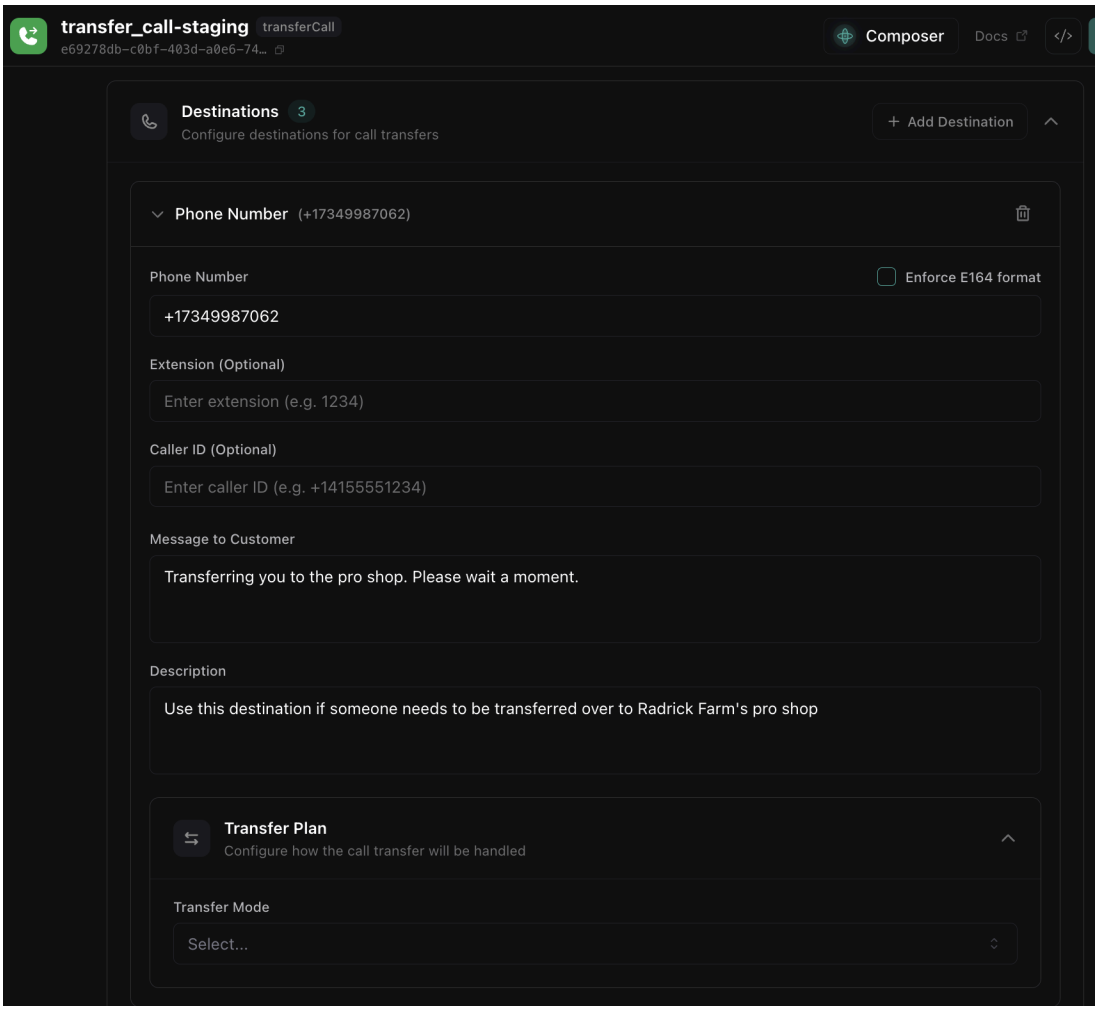
The System Prompt is designed to transfer calls to specific departments (e.g., the Pro Shop, Restaurant, or Agronomy). However, the prompt simply outputs a command like: `<call transfer_call-staging tool with destination='pro_shop'>`.

For this to actually work, Vapi must know what phone number `pro_shop` maps to.

Step 3A: Adding Destinations in Vapi

1. In the Vapi dashboard sidebar, click on **Tools**.
2. Find and open the **`transfer_call-staging`** tool.
3. Scroll down to the destination configurations and click **Add Destination**. You can add as many as the facility requires.
4. Fill out the fields precisely:
    - **Number:** Must be in E.164 format (e.g., `+17202304702`).
    - **Message:** This is what the AI says right before transferring. (e.g., *"Transferring you to the Agronomy and maintenance team. Please wait a moment."*)
    - **Description:** This helps the AI map the prompt command to this tool. (e.g., *"Whenever the caller requests or requires a transfer to the Agronomy and maintenance team as per the criteria defined in the prompt."*)
    - **Transfer Plan:** Set this to **Blind Transfer** by default.

- **Extension (Optional):** If the client's phone tree requires a dialpad digit (e.g., "Press 2 for events"), put the extension here.



Step 3B: Syncing with the Prompt

CRITICAL RULE: Any destination you configure in the Vapi Tool UI MUST exactly match the allowed destinations written in the System Prompt's `<core-shell>` or `<logic-module>`. If Vapi has a destination named `agronomy_team` but the prompt tells the AI to use `agronomy`, the transfer will fail and the call will drop. Keep them identical.

---

4. Setting up Background Noise Correction for New Facilities

When a new facility goes live, the AI can sometimes be overly sensitive to background noise on the caller's end (e.g., wind, car noise, or heavy breathing), causing the AI to interrupt itself prematurely.

We fix this by increasing the number of words required to interrupt the assistant.

Step 4A: The API Patch

Currently, this is not exposed in the Vapi UI, so it requires an API call. **You will need to ask Jim to execute this.** Provide Jim with the Assistant ID (found in the Vapi URL for the facility) and ask him to run the following cURL command:

```shell
curl -X PATCH "https://api.vapi.ai/assistant/assistant-id-123" \
  -H "Authorization: Bearer your-auth-token-here" \
  -H "Content-Type: application/json" \
  -d '{
    "numWordsToInterruptAssistant": 3
  }'
```

Note: Setting this to `3` ensures a stray cough or short background noise doesn't stop the AI from speaking.

---

5. Enabling SMS for Facilities via Twilio and Vapi Tool Addition

For non-integrated facilities (or integrated facilities utilizing SMS confirmations), we need to ensure the AI has the physical capability to send text messages.

Step 5A: Twilio Senders Pool (Jim's Task)

You will need Jim's help for this. Jim must log into Twilio and add the new facility's phone number to our Sender Pool for our low-volume messaging service (A2P compliance).

Step 5B: Vapi Tool Mapping

Once Jim confirms the number is in the Twilio pool, you must explicitly give the AI Agent access to the SMS tool.

1. Go to the specific facility's workspace in **Vapi**.
2. Open the `Facility Receptionist (staging)` assistant configuration page.
3. Scroll down to the **Tools** section.
4. Click the dropdown and select the **`send_sms-staging`** tool. *(Note: This is NOT done by default upon creation, so you must remember to add it).*
5. Click **Publish**.

---

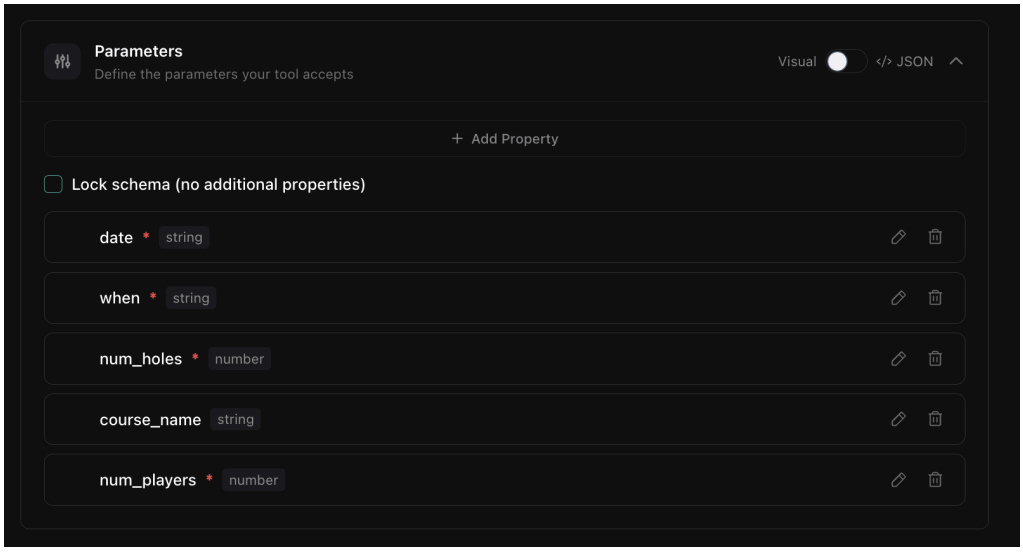
6. Ensuring Tool Configurations for Tee Time Tools

For Fully-Integrated facilities, the AI uses two critical tools to interact with the tee sheet: `get-available-tee-times-staging` and `book-tee-time-staging`.

Due to a current provisioning quirk, these tools do not automatically import all the necessary parameters when Jim provisions a new account. You must manually add them.

Step 6A: Configuring the Get Availability Tool

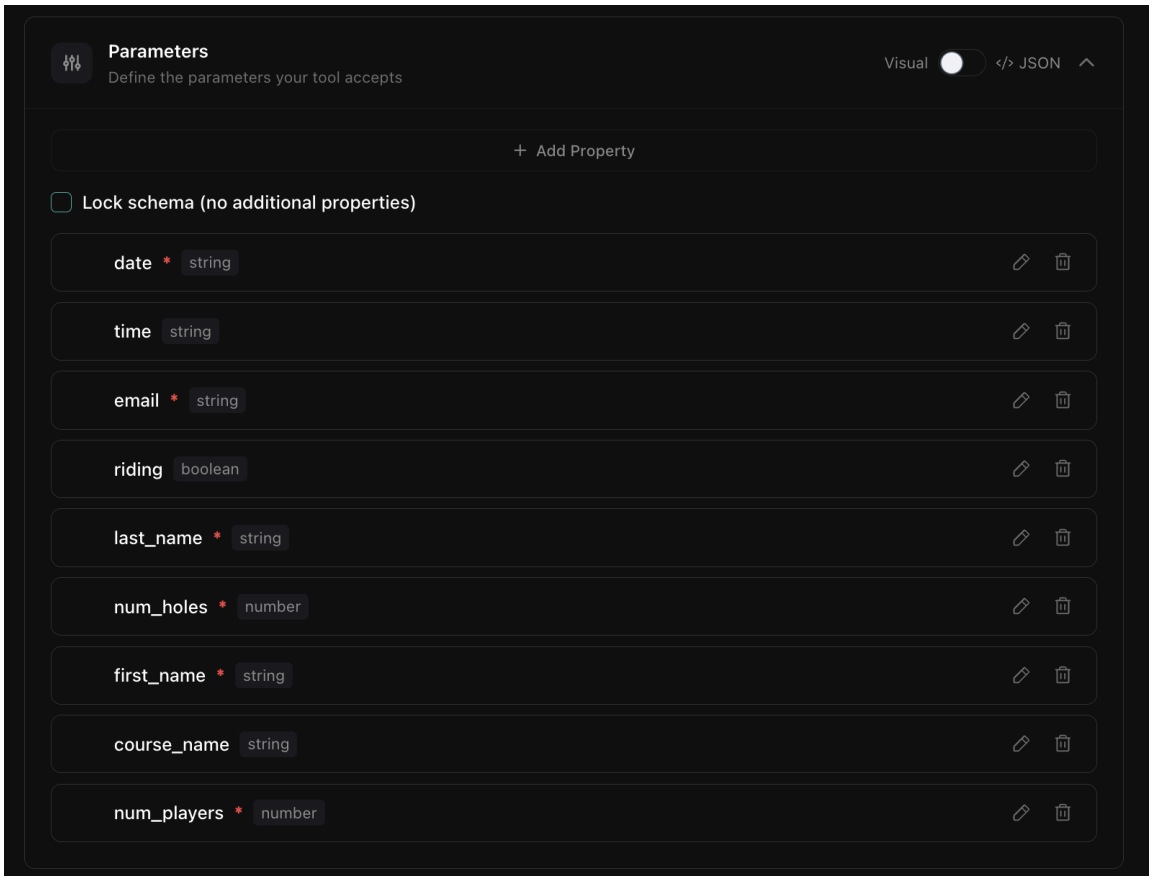
1. In Vapi, click on **Tools** in the sidebar.
2. Open **`get-available-tee-times-staging`**.
3. Scroll to the Parameters list. If `course_name` is missing, click **Add Parameter**.
4. Set the Parameter Name to `course_name`.
5. Set the Type to **String**.
6. Mark it as **Optional** (Toggle the required switch off).
7. Save.



Step 6B: Configuring the Booking Tool

1. In Vapi, go back to **Tools**.
2. Open **`book-tee-time-staging`**.
3. Check the Parameters list. You need to ensure both `course_name` and `riding` exist.
4. **If `course_name` is missing:**
    - Add Parameter -> Name: `course_name` -> Type: **String** -> Optional.
5. **If `riding` is missing:**
    - Add Parameter -> Name: `riding` -> Type: **Boolean** -> Optional. *(Note: The 'riding' parameter is strictly required for the AI to enforce cart-path/mandatory-cart rules on weekends, so it must be present as a boolean).*

6. Save all changes. Ensure you go back to the Assistant and hit **Publish** to solidify the tool state.



Final Note: If you run into an issue where an agent is failing a tool call or hanging up, 90% of the time it is a mismatch between the Prompt logic, the Variables initialized at the top of the prompt, or the Tool Parameters defined in Vapi. Always check the call logs in Vapi first to see the exact error response.

Good luck, and please lean on this document heavily!